

Homework 4

RISC-V Assembly Programming

For this assignment you'll write a program in RISC-V assembly that can be executed on this RISC-V simulator: <https://ripes.me/>

Step 1

Write a program `powHardcoded.S` that prints the string, "This program prints 5⁶..." and then prints the number 15625.

The program should define the string in the `.data` section and the rest of the program code in the `.text` section. Everything is hard-coded. It doesn't actually compute 5⁶ but rather just prints 15625, hard-coded into the assembly code.

For printing the string you may use system call 4 (PrintString) or system call 64 (Write). If you use system call 64 you should pass a 1 as the first parameter to specify that the output should go to the console (stdout). For printing the int you can use any system call you want.

The program should end with exit code 0. You may use system call 10 or 93.

Step 2

Write a program `pow.S` that prints the string, "This program computes 4⁷..." and then goes on to actually compute 4⁷, to print the result. You should use your `powHardcoded.S` program as a starting point. For this program, 4 and 7 will be hard-coded into the assembly, but the result will be computed.

Similarly to step 1, use the appropriate system calls to print the string, the output number, and to correctly exit with status 0. Setup the code with the appropriate `.data` and `.text` sections.

For computing 4⁷ you should define a function `pow(a, b)` that implements the following algorithm.

```
pow(a, b) {
    ans = 1
    while(b != 0) {
        ans = ans * a
        b = b - 1
    }
    return ans
}
```

To call your function use the `jal` instruction. To pass arguments to the function you should use the `a0` and `a1` registers. To return the final answer, you should use the `a0` register.

After you have the code working for $a=4$ and $b=7$, change the values to $a=9$, $b=8$ and make sure the program still works correctly. Of course, you should use a calculator to compute 9^8 yourself when checking that your assembly code works.

Step 3

Write a program `powRecursive.S` that prints the string, “This program computes 2^3 ...” computes 2^3 **recursively**, and then prints the result. Again, for this program, 2 and 3 will be hard-coded into the assembly, but the result will actually be computed.

Similarly to step 1 and 2, use the appropriate system calls to print the string, the output number, and to correctly exit with status 0. Setup the code with the appropriate `.data` and `.text` sections.

For computing 2^3 you should define a function `pow(a, b)` that implements the following recursive algorithm.

```
pow(a, b) {
    if (b == 0) {
        return 1
    }

    if (a == 0) {
        return 0
    }

    ans = a * pow(a, b-1)
    return ans
}
```

Similarly to step 3, use the `jal` instruction to call your `pow` function. To pass arguments to the function you should use the `a0` and `a1` registers. To return the final answer, you should use the `a0` register.

Calling this function from `main` is fine, but this function itself is recursive. This means that `pow()` is called inside itself and intermediate values like `a0` and `a1` will be overwritten. To avoid this problem, use the function call stack by utilizing the stack pointer `sp`

1) Increment the stack pointer to make room for the values of `a0`, `a1`, and `ra` *before calling* `pow()` inside the `pow()` function. Use the `sw` instruction to put the values of `a0`, `a1`, and `ra` into memory “on the stack” based on where the `sp` is pointing.

2) Call `pow()`

3) When `pow()` returns, move the value out of `a0` into a different register (recommended: any of the “temporaries”). Then, use the `lw` instruction to retrieve the old values of `a0`, `a1`, and `ra` from “the stack” (memory) based on where `sp` is pointing. Finally, return `sp` to its original position.

After doing all of this, you should have the value of `pow(a, b-1)` *after the recursive call has been finished* in a temporary register (e.g., `t2`).

`ans = a * pow(a, b-1)`



Finally, compute the multiplication and store the result in `a0` to be returned.

After you have the code working for `a=2` and `b=3`, change the values to `a=9`, `b=8` and make sure the program still works correctly.

Helpful Stuff and Useful Tidbits

- This website: <https://www.recursionvisualizer.com/> which accepts python code.
- Consider using the "single-cycle" processor mode when debugging (click the little processor icon in `ripes`).
- The provided "riscv-card.pdf" cheat sheet does not include some operations. Notably absent are `mul` and `div` which do multiplication and division respectively. These operations **are** supported in `ripes`.
- The copy/paste features in `ripes` are not great. I recommend using the keyboard shortcuts (`ctrl+a` to select all, `ctrl+c` to copy, `ctrl+v` to paste, and `ctrl+x` to cut) instead of the mouse right-click context-menu.

What To Submit

You should submit (on canvas) all three of your assembly programs as text files.

`powHardcoded.S`

`pow.S`

`powRecursive.S`

Bonus Stuff [ungraded]

Assuming you've had so much fun with this assignment that you cannot contain yourself you might want to try to implement the "fast exponentiation" algorithm in assembly.

```
powFast(a, b) {  
    if (b == 0) {  
        return 1  
    }  
    else if (b % 2 == 0){  
        return powFast(a, b / 2) * powFast(a, b / 2)  
    }  
    else {  
        return a * powFast(a, b / 2) * powFast(a, b / 2)  
    }  
}
```

Ideally we would implement an assembly program that allows for the user to enter their chosen value of a and b. Unfortunately this functionality is not working in ripes. Please feel free to fix it.

<https://github.com/mortbopet/Ripes/issues/304>